```
public class FlightDetailsSteps {

    String flightNumber;
    Flight matchingFlight;

    @Given("^I need to know the details of flight number (.*)$")
    public void flight_number(String flightNumber) throws Throwable {
        this.flightNumber = flightNumber;
    }

    @When("^I request the details about this flight$")
    public void request_flight_details() throws Throwable {
        FlightStatusClient client = new FlightStatusClient();
        matchingFlight = client.findByFlightNumber(flightNumber);
    }

    @Then("^I should receive the following:$")
    public void verify_details(DataTable flightDetails) throws Throwable {
        flightDetails.diff(newArrayList(matchingFlight));
    }
}
```

Note what flight number you're looking for.

1 Write a simple web service client to access the web service.

2 Compare the expected data with what the web service returned.

The automation code is relatively simple. You use a class written for your tests to access the web service and retrieve the result in the form of a Java object 1.

To verify the outcomes, you use Cucumber's `DataTable` class 2, which provides a convenient `diff()` method to compare the contents of a table with the corresponding field values from a list of Java objects, using column headers as field names. This is a concise way of checking that the `Flight` object that you retrieved from the web service matches the values described in the scenario.

When it comes to automating the BDD scenarios that verify services, it's common to use exactly the same libraries that are used to access them in the production code. This makes sense, because the automated steps act as examples for other developers to follow when reusing the services. In this example, you're using JAX-RS and Jersey, the standard Java library used to implement web services and web service clients:

```
public class FlightStatusClient {

    private final String BASE_URL = "http://localhost:8080/rest/flights";

    public Flight findByFlightNumber(String flightNumber) {
        Client client = ClientBuilder.newClient();
        WebTarget webTarget = client.target(BASE_URL).path(flightNumber);
        return webTarget.request().buildGet().invoke(Flight.class);
    }
}
```

The URL of the web service.

Create a new web service client.

Specify the resource path you're invoking.

Retrieve the result and convert it to a Flight object.

This level of feature is sufficient and appropriate for many scenarios. But some of the scenarios you write to describe a service may have a more technical audience than more business-focused scenarios. In some contexts, for example, if you're implementing